Data Structures for Placements — Day 1: Arrays

DSA Prep Notes

oday

Day 1: Complexity Analysis & Arrays

1. Why Data Structures Matter for Placements

Every coding round tests: can you pick the right data structure for a problem, and can you reason about its efficiency? Interviewers care less about "does it work" and more about "why did you choose this, and what's its cost."

2. Time & Space Complexity (Big-O)

Big-O describes the **upper bound** — worst case growth rate of time/space as input size `n` grows.

Notation	Name	Example
O(1)	Constant	Array index access
O(log n)	Logarithmic	Binary search
O(n)	Linear	Single loop scan
O(n log n)	Linearithmic	Merge sort
O(n^2)	Quadratic	Nested loops (bubble sort)
O(2^n)	Exponential	Recursive Fibonacci (naive)

Related notations:

- **Big-Omega (Ω)** — best case / lower bound
- **Big-Theta (Θ)** — tight bound (when best = worst, i.e. average case)

Rules of thumb:

- Drop constants: O(2n) becomes O(n)
- Drop lower-order terms: O(n^2 + n) becomes O(n^2)
- Nested loops over the same input → multiply: O(n) x O(n) = O(n^2)
- Sequential loops → add, then keep the dominant term: O(n) + O(n^2) becomes O(n^2)

Space complexity counts extra memory used (auxiliary space), not counting the input itself.

3. Arrays — Theory

An **array** is a collection of elements stored in **contiguous memory locations**, accessed via an index.

Key properties:

- Fixed size (in C++ static arrays) — size decided at compile/allocation time
- **Random access** in O(1): `address = base_address + (index * size_of_element)`
- All elements are of the same data type

Types:

- **Static array**: `int arr[5];` — size fixed at compile time
- **Dynamic array**: `vector<int>` in C++ — resizable at runtime

Complexity of Array Operations

Operation	Time Complexity
Access by index	O(1)
Search (unsorted)	O(n)
Search (sorted, binary search)	O(log n)
Insertion at end (vector)	O(1) amortized
Insertion at beginning/middle	O(n) — must shift elements
Deletion at end	O(1)
Deletion at beginning/middle	O(n) — must shift elements

How dynamic arrays (vector) grow: when capacity is full, a vector typically **doubles** its capacity — allocates new memory, copies old elements, frees old memory. This is why `push_back` is O(1) *amortized*: most insertions are O(1), occasional resize is O(n), but averaged over many insertions it works out to O(1).

4. Full C++ Implementation — Basic Operations

See `01_array_basics.cpp` in this folder for the complete, tested code covering: traversal, insertion, deletion, linear search, binary search, reversal (iterative + recursive), max/min finding, and dynamic array (vector) usage.

Solved Problems — Deep Dive

Problem 1: Reverse an Array In-Place

Goal: O(n) time, O(1) space, no extra array.

Two-pointer swap from both ends toward the center. A recursive version also exists but costs O(n) space on the call stack.

Dry run on [1,2,3,4,5]: start=0,end=4 → swap → [5,2,3,4,1]; start=1,end=3 → swap → [5,4,3,2,1]; start=2,end=2 → stop.

Edge cases: empty array (loop never runs, safe), single element (no swap needed), odd-length arrays have a middle element that never swaps — confirm loop condition is `start < end`, not `<=`.

Problem 2: Missing Number (array has 1 to n, one missing)

Brute force: for each number 1..n, linear search if present — O(n^2).

Sum approach — O(n): `expectedSum = n*(n+1)/2`; missing = expectedSum − actualSum. **Must use `long long`** to avoid integer overflow for large n.

XOR approach — O(n), no overflow risk: XOR all numbers 1..n, then XOR with all array elements. Since `a XOR a = 0` and XOR is commutative/associative, every present number cancels out, leaving only the missing number.

Dry run: n=5, arr=[1,2,4,5] (missing 3) → xorAll = 1, xorArr = 2 → result = 1 XOR 2 = 3. Correct.

Limitation: both approaches only work if **exactly one** number is missing.

Problem 3: Kadane's Algorithm (Maximum Subarray Sum)

One of the most frequently asked array problems in interviews.

Core idea: at each index, decide whether to extend the previous subarray or start fresh.

```
maxEndingHere = max(arr[i], maxEndingHere + arr[i])
maxSoFar = max(maxSoFar, maxEndingHere)
```

Dry run on [-2,1,-3,4,-1,2,1,-5,4]: the running maxEndingHere goes -2, 1, -2, 4, 3, 5, 6, 1, 5 — and maxSoFar peaks at **6**, corresponding to subarray [4,-1,2,1].

Critical bug to avoid: initialize `maxEndingHere` and `maxSoFar` with `arr[0]`, **not 0**. Initializing with 0 breaks the all-negative-array case (e.g. [-3,-1,-4] should return -1, the least negative element, not 0).

A variant of this algorithm also tracks and returns the start/end indices of the maximum subarray — useful for follow-up interview questions.

Problem 4: Move All Zeroes to End (preserve order of non-zeros)

Brute force: copy non-zeros to a new array, then fill zeros — O(n) time, O(n) space.

Optimal — two pointer, O(n) time, O(1) space: maintain an `insertPos` pointer; whenever a non-zero element is found, swap it into `insertPos` and advance.

Dry run on [0,1,0,3,12]: result is **[1,3,12,0,0]** — order preserved, zeros pushed to the end.

Edge cases: no zeros present (self-swaps occur but are harmless), all zeros (no swaps happen, array unchanged — correct).

Problem 5: Two Sum

Brute force — O(n^2): nested loop checking all pairs.

Optimal — Hashing, O(n) time, O(n) space: for each element, check if its complement (target − current) was already seen in a hash map; if yes, return the pair of indices. Only one pass needed.

Alternative — Sorted + Two Pointer, O(n log n) time, O(1) extra space: sort the array, then move pointers from both ends based on whether the current sum is above or below target.

Key interview trap: sorting destroys original index order. Use hashing when the question asks for **indices**; sorting + two pointer is fine when it asks for **values** only, and it saves space compared to hashing.

Edge cases: no valid pair exists, duplicate values (handled correctly since we check `seen` *before* inserting the current element, avoiding self-pairing), negative numbers (handled fine by both approaches).

Summary Table — Day 1

Problem	Optimal Approach	Time	Space
Reverse Array	Two Pointer	O(n)	O(1)
Missing Number	XOR	O(n)	O(1)
Maximum Subarray Sum	Kadane's Algorithm	O(n)	O(1)
Move Zeroes to End	Two Pointer	O(n)	O(1)
Two Sum	Hashing	O(n)	O(n)

Code files for today: `01_array_basics.cpp`, `02_placement_problems.cpp` — both compiled and tested.

Next session (Day 2): Strings + Searching/Sorting basics.